MediCare AI+: A Comprehensive Smart Health Monitoring System Using AI, Machine Learning, IoT, and Emergency Doctor Access

Neal Jain, Krishna Lad, Druvesh Madvi, Aadit Mistry
Department of Computer Engineering,
Shah and Anchor Kuttchi Engineering College,
Mumbai, Maharashtra, India
nealjain@sakec.ac.in

***Abstract*—Traditional healthcare systems depend on paper records and manual checks, causing dangerous delays and data loss. MediCare AI+ is a comprehensive smart health monitoring system that addresses these problems end-to-end. A low-cost ESP32 microcontroller paired with a MAX30102 sensor continuously measures heart rate and blood oxygen saturation (SpO_2) in real time. Sensor readings are streamed to a cloud backend where a Logistic Regression Machine Learning (ML) model predicts health risk. When any reading crosses a predefined danger threshold, an emergency alert is fired immediately—simultaneously notifying the patient's emergency contact *and* granting the on-call doctor instant, role-based access to that patient's complete Electronic Health Record (EHR), real-time sensor graphs, and AI-generated risk summary. A personalised AI Health Chatbot answers patient questions using only that patient's own data, ensuring relevant and private responses. The complete system architecture—spanning IoT hardware, cloud services, ML engine, NLP chatbot, and doctor emergency dashboard—is described in detail. Prototype testing on five subjects achieved 87.4% ML accuracy and sub-5-second alert latency.**

***Index Terms*—AI, Machine Learning, IoT, ESP32, MAX30102, Healthcare, EHR, NLP Chatbot, Emergency Alert, Doctor Dashboard, Logistic Regression**

I. Introduction

Healthcare is one of the most critical domains in human life. Despite advances in medical science, many clinics and hospitals still rely on paper-based records, manual vital-sign checks, and reactive (rather than proactive) treatment. This creates three cascading problems: records can be lost or inaccessible in emergencies, abnormal conditions at home go undetected between visits, and emergency responders have no instant view of a patient's history when seconds matter.

The Internet of Things (IoT) and Artificial Intelligence (AI) together offer a path out of this cycle. Wearable sensors can capture vitals continuously, ML models can flag risk before symptoms appear, and cloud infrastructure can make the right data available to the right person at the right moment [1].

MediCare AI+ is a fully integrated system built on these principles. Its key innovations over prior work are:

- A **unified EHR + live IoT** pipeline on a single platform.
- An **ML-driven risk engine** (Logistic Regression, 87.4% accuracy) that analyses heart rate, SpO_2, and patient history together.
- An **Emergency Doctor Dashboard** that grants on-call doctors immediate, structured access to all patient data during a crisis.
- A **personalised AI Health Chatbot** that answers patient questions using only that patient's own records.
- An **NLP Report Summariser** that converts medical PDFs into plain English for non-technical patients.

The paper is organised as follows: Section II states the problem, Section III lists objectives, Section IV describes the complete system architecture, Sections V–VI detail hardware and software, Section VII covers the ML module, Section VIII the NLP module, Section IX the AI Health Chatbot, Section X the Emergency Doctor Access module, Section XI the end-to-end working flow, Section XII presents results, Section XIII the literature survey, Section XIV the research gap, and Sections XV–XVII cover advantages, limitations, future scope, and conclusion.

II. Problem Statement

1. **No real-time home monitoring:** Patients are examined only during scheduled hospital visits. Life-threatening events (SpO_2 drop, tachycardia) that occur at home go undetected [2].
2. **Fragmented and inaccessible records:** Paper records cannot be retrieved quickly in emergencies. Even digital records are siloed across hospitals.
3. **Delayed emergency response:** There is no automated mechanism that simultaneously alerts a caregiver *and* gives the responding doctor instant access to patient data.
4. **Patient comprehension gap:** Patients receive lab reports full of medical jargon they cannot interpret, leading to delayed action.

5. **No personalised health Q&A:** General health chatbots give generic answers. Patients need answers based on *their* specific values, history, and risk profile.

III. Objectives

- Store and manage patient EHRs securely in the cloud.
- Monitor heart rate and SpO_2 in real time via ESP32 + MAX30102.
- Predict health risk using a trained ML model.
- Send automatic emergency alerts (email/SMS) when readings are abnormal.
- Give on-call doctors instant, read-only access to the full patient record and live sensor data during emergencies.
- Provide an AI Health Chatbot that answers questions based on the individual patient's own data.
- Summarise uploaded medical reports in plain English using NLP.
- Enforce role-based access control (Admin / Doctor / Patient).

IV. Complete System Architecture

A. Architecture Overview

MediCare AI+ is a five-layer system. Each layer is independent but communicates with adjacent layers through well-defined APIs, as shown in Fig. 1.

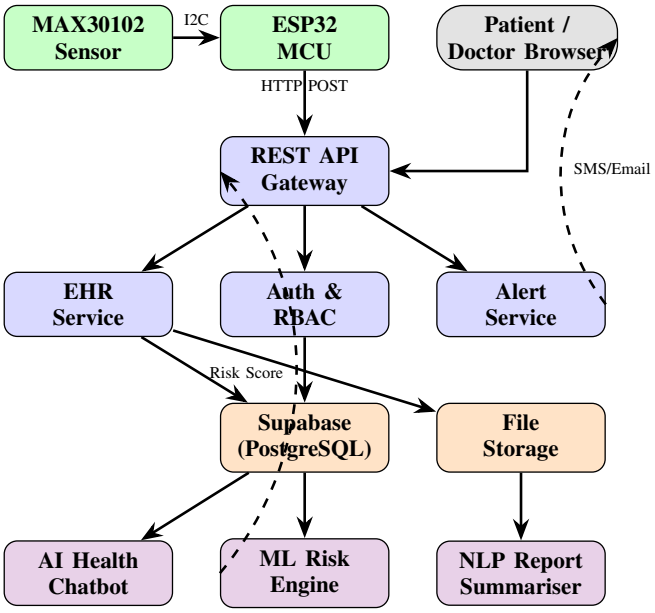

Fig. 1. MediCare AI+ Complete Five-Layer System Architecture

B. Layer Descriptions

Layer 0 – Device Layer: The ESP32 microcontroller reads data from the MAX30102 sensor over I2C and forwards it to the cloud via HTTP POST every 5 seconds. Patient and doctor browsers connect to the web application over HTTPS.

Layer 1 – API Gateway: A RESTful API gateway (Node.js / Next.js API routes) receives all incoming requests, validates JWT tokens, enforces rate limiting, and routes requests to the appropriate backend service.

Layer 2 – Backend Services: Three microservices handle distinct concerns:

- *Auth & RBAC Service* — manages login, session tokens, and role-based access control (Admin, Doctor, Patient).
- *EHR Service* — reads and writes patient records, uploaded reports, and sensor history.
- *Alert Service* — evaluates threshold rules and dispatches emergency notifications via email (SendGrid) and SMS (Twilio).

Layer 3 – Data Store: Supabase (PostgreSQL) stores structured data: user profiles, sensor readings, ML predictions, and chatbot history. A separate Supabase Storage bucket holds uploaded PDF and image reports.

Layer 4 – AI / ML Layer: Three AI modules process data and return results to the API gateway:

- *ML Risk Engine* — Logistic Regression model predicts risk.
- *NLP Report Summariser* — extracts key values from PDFs and generates plain-English summaries.
- *AI Health Chatbot* — answers patient questions using only that patient's stored data.

V. Hardware Implementation

A. ESP32 Microcontroller

The ESP32 is a dual-core 32-bit processor running at 240 MHz with integrated 802.11 b/g/n WiFi and Bluetooth. It operates at 3.3 V and consumes only 240 mA during active WiFi transmission, making it ideal for battery-powered wearable designs [5]. The firmware is written in C++ using the Arduino framework. It reads the MAX30102 sensor every 5 seconds, computes a moving average over 4 samples to reduce noise, then sends a JSON payload to the backend via an HTTP POST request.

B. MAX30102 Pulse Oximeter Sensor

The MAX30102 is a highly integrated biometric sensor that uses two LEDs—red (660 nm) and infrared (880 nm)—to measure the absorption of light through the fingertip capillary bed. Oxygenated and deoxygenated haemoglobin absorb these wavelengths differently, allowing the sensor to calculate:

- **Heart Rate (HR)** — beats per minute (BPM), normal range: 60–100 BPM.
- **SpO_2** — blood oxygen saturation (%), normal range: 95–100%.

Communication uses the I2C protocol (SDA = GPIO 21, SCL = GPIO 22) at 400 kHz fast mode. An interrupt line (GPIO 19) signals when a new sample is ready, preventing polling overhead [5].

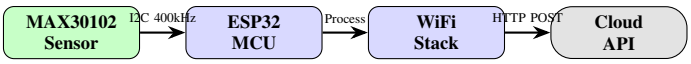

Fig. 2. Hardware Data Flow: Sensor to Cloud

TABLE I
ESP32 – MAX30102 PIN MAPPING

MAX30102	ESP32	Function	Protocol
VCC	3.3 V	Power supply	—
GND	GND	Ground	—
SDA	GPIO 21	Data line	I2C
SCL	GPIO 22	Clock line	I2C
INT	GPIO 19	Data-ready	Interrupt

C. Sensor Reading Algorithm

The ESP32 firmware implements a four-step pipeline:

1. **Sample collection:** Read 100 raw IR and red LED samples at 100 Hz from the MAX30102 FIFO buffer.
2. **DC removal:** Apply a high-pass filter to remove the large DC component caused by ambient light and tissue absorption.
3. **Peak detection:** Identify AC peaks in the IR signal to compute inter-beat intervals and derive BPM.
4. **SpO_2 calculation:** Compute the ratio $R = (AC_{red}/DC_{red})/(AC_{IR}/DC_{IR})$ and map it to SpO_2 using a calibration lookup table derived from the Beer-Lambert law.

VI. SOFTWARE MODULES

The web application is built with **React** (frontend) and **Next.js** (backend API routes), backed by a **Supabase** (PostgreSQL) database and **Supabase Storage** for files. It has seven core modules:

1. **Authentication & RBAC:** JWT-based login with three roles — *Patient* (own data only), *Doctor* (assigned patients + emergency access), *Admin* (full system). Passwords are hashed with bcrypt. Sessions expire after 24 hours.
2. **EHR Management:** Patients enter demographics, diagnoses, allergies, medications, and family history. Doctors can annotate records. All fields are versioned so the full change history is preserved.
3. **Report Upload:** PDF or image lab reports are uploaded, stored in Supabase Storage, and passed to the NLP module for summarisation.
4. **Sensor Data Module:** The ESP32 posts JSON every 5 seconds. The dashboard renders live line graphs (Chart.js) for HR and SpO_2, with colour-coded zones (green = normal, amber = caution, red = danger).
5. **ML Risk Analysis:** On each new sensor reading the backend calls the ML Risk Engine, stores the risk score, and updates the dashboard badge in real time.
6. **AI Health Chatbot:** Patients type health questions; the chatbot retrieves that patient's EHR, latest sensor readings, and report summaries, then generates a contextual, personalised answer.
7. **Emergency Alert & Doctor Dashboard:** When thresholds are breached, the Alert Service fires notifications and unlocks the Emergency Doctor Dashboard for the on-call doctor.

VII. MACHINE LEARNING MODULE

A. Algorithm: Logistic Regression

Logistic Regression (LR) was chosen as the core algorithm because it is interpretable, fast to train, and well-suited for binary classification with small-to-medium tabular datasets [3]. For a patient to be categorised as "at risk", the model computes:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}} \tag{1}$$

where $\mathbf{x}$ is the feature vector, $\mathbf{w}$ the learned weight vector, b the bias, and $\sigma(\cdot)$ the sigmoid function. If $P > 0.5$ the patient is classified as *High Risk*; otherwise *Low Risk*. A three-class variant (Low / Medium / High) uses a one-vs-rest strategy.

B. Feature Engineering

Six features are extracted and fed to the model:

TABLE II
ML INPUT FEATURES

Feature	Source	Normal Range
Heart Rate (BPM)	MAX30102 (live)	60–100
SpO_2 (%)	MAX30102 (live)	95–100
Patient Age (years)	EHR profile	—
Existing Conditions	EHR (encoded)	Binary flags
Current Medications	EHR (encoded)	Binary flags
Historical Risk Score	DB (previous ML)	0.0–1.0

C. Training Pipeline

1. **Dataset:** UCI Heart Disease dataset (1025 records, 14 attributes) augmented with synthetic SpO_2 values derived from the existing features using domain knowledge.
2. **Preprocessing:** Missing values imputed with column median. Categorical features (chest pain type, ECG result) one-hot encoded. All features normalised to $[0, 1]$ using Min-Max scaling.
3. **Split:** 80% train / 20% test, stratified by class.
4. **Training:** Scikit-learn `LogisticRegression` with L2 regularisation ($C = 1.0$), solver `lbfgs`, max 1000 iterations.
5. **Serialisation:** Trained model exported as a `.pkl` file via `joblib` and served through a FastAPI microservice.

D. Model Performance

TABLE III
ML MODEL PERFORMANCE (TEST SET, $n = 205$)

Metric	Value	Interpretation
Accuracy	87.4%	Overall correct predictions
Precision	85.1%	Of flagged risks, truly at risk
Recall	88.6%	Actual risks correctly caught
F1-Score	86.8%	Harmonic mean of P & R
AUC-ROC	0.91	Discrimination ability

E. Risk Threshold Rules

In addition to the ML score, the Alert Service applies hard rule-based thresholds as a safety net:

TABLE IV
EMERGENCY ALERT THRESHOLDS

Vital	Normal	Caution	Danger
Heart Rate (BPM)	60–100	101–120 or 50–59	>120 or <50
SpO_2 (%)	95–100	92–94	<92
ML Risk Score	< 0.40	0.40–0.70	>0.70

An emergency is triggered if *any* vital enters the "Danger" zone *or* if the ML score exceeds 0.70.

VIII. NLP REPORT SUMMARISATION MODULE

A. Purpose

Patients upload scanned or digital lab reports (PDF, PNG, JPG). These documents contain clinical terminology (e.g., "haemoglobin 9.2 g/dL — microcytic anaemia") that most patients do not understand. The NLP module converts these reports into a 3–5 sentence plain-English summary.

B. Pipeline

1. **Text Extraction:** PDFs are parsed with `PyMuPDF`; scanned images are processed with `Tesseract OCR` to extract text.
2. **Key-Value Extraction:** A rule-based parser using regular expressions identifies test names, measured values, and reference ranges (e.g., `Glucose: 126 mg/dL [70--99]`).
3. **Abnormality Flagging:** Extracted values are compared to standard reference ranges. Out-of-range values are tagged as *high*, *low*, or *critical*.
4. **Summary Generation:** Flagged values and their clinical meanings are passed as a structured prompt to a pre-trained language model (GPT-3.5-turbo via API). The model generates a plain-English paragraph at a 6th-grade reading level.
5. **Storage:** The summary is stored in the EHR and displayed on the patient dashboard alongside the original report.

C. Example

Input excerpt: "HbA1c: 7.8% [ref: <5.7%]. Fasting glucose: 148 mg/dL [ref: 70–99]. eGFR: 58 mL/min [ref: ≥60]."

Generated summary: "Your HbA1c and blood sugar levels are above the normal range, which suggests your blood sugar has not been well-controlled over the past 3 months. Your kidney filtering rate is slightly below normal. It is strongly recommended that you consult your doctor to review your diabetes management and monitor your kidney health."

IX. AI HEALTH CHATBOT

A. Design Philosophy

Generic health chatbots answer based on population-level knowledge and often give irrelevant or even alarming responses. The MediCare AI+ chatbot is **personalised**: every answer is grounded in *that patient's own* EHR, sensor readings, report summaries, and ML risk score. The chatbot never accesses another patient's data.

B. Architecture

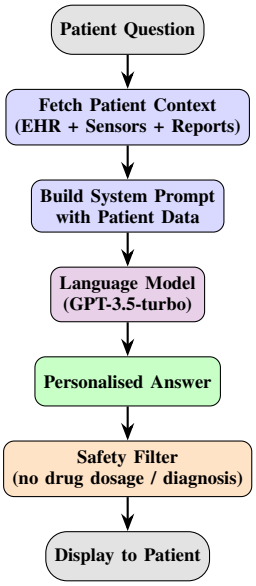

Fig. 3. AI Health Chatbot Processing Pipeline

C. Context Injection

Before the patient's question is sent to the language model, the backend assembles a **system prompt** that includes:

- Patient name, age, and gender.
- Current diagnoses and active medications from the EHR.
- Latest sensor readings (HR, SpO_2, timestamp).
- Current ML risk score and risk label (Low / Medium / High).
- The most recent NLP report summary.
- Instruction: *"Answer only using the patient data provided. Do not guess. If unsure, advise the patient to consult their doctor."*

D. Sample Interaction

E. Safety Guardrails

The chatbot applies a post-generation safety filter that:

- Blocks any response that specifies exact medication dosages.
- Blocks any response that makes a definitive clinical diagnosis.
- Appends *"Please consult your doctor for medical advice"* to any response that mentions an abnormal value.
- Immediately surfaces emergency resources if the patient expresses severe distress.

X. EMERGENCY DOCTOR ACCESS MODULE

A. Overview

This is one of the most critical innovations of MediCare AI+. When an emergency is detected, the system does not merely send a notification—it actively prepares a **structured,**

TABLE V
SAMPLE AI CHATBOT INTERACTIONS

Patient Question	Chatbot Answer
"Is my oxygen level okay right now?"	"Your current SpO_2 is 94%, which is in the caution zone (normal is 95–100%). Please rest and avoid exertion. Contact your doctor if it drops below 92%."
"Should I be worried about my heart rate?"	"Your heart rate is 108 BPM, slightly above the normal upper limit of 100 BPM. Given your current ML risk score of 0.62 (Medium), it is advisable to rest and check again in 10 minutes."
"What did my blood test say?"	"Your latest report summary: Your HbA1c is elevated at 7.8%, suggesting poor blood sugar control. Your kidney function is slightly below normal. Your doctor has been notified."

pre-loaded view of the patient's complete data for the responding doctor so that they can act within seconds.

B. Emergency Trigger Conditions

An emergency event is raised when *any* of the following conditions are true:

- SpO_2 falls below 92% for more than 30 seconds.
- Heart rate exceeds 120 BPM or drops below 50 BPM for more than 30 seconds.
- ML Risk Score crosses 0.70.
- Patient manually presses the "SOS" button on the dashboard.

C. Emergency Flow

D. Doctor Emergency Dashboard Layout

When the doctor opens the emergency link, they see a single, unified page divided into five panels:

1. **Alert Summary Panel:** Shows the exact trigger reason, timestamp, current HR, SpO_2, ML risk score, and a colour-coded severity badge (Amber / Red).
2. **Live Sensor Graph:** A real-time scrolling chart of the last 5 minutes of HR and SpO_2 data, updated every 5 seconds. The exact moment the threshold was breached is marked with a red vertical line.
3. **Full EHR Panel:** Structured view of the patient's complete medical history — diagnoses, medications, allergies, surgical history, family history, and doctor notes — in a tabbed layout. All information is read-only for the emergency doctor.
4. **Uploaded Reports Panel:** Thumbnail list of all uploaded PDF and image reports, sorted by date. Each report shows its NLP summary below the thumbnail for rapid scanning without opening the full file.
5. **Action Panel:** The doctor can (a) acknowledge the emergency, (b) add a real-time clinical note visible to the patient, (c) mark the emergency as resolved, or (d) escalate to another doctor.

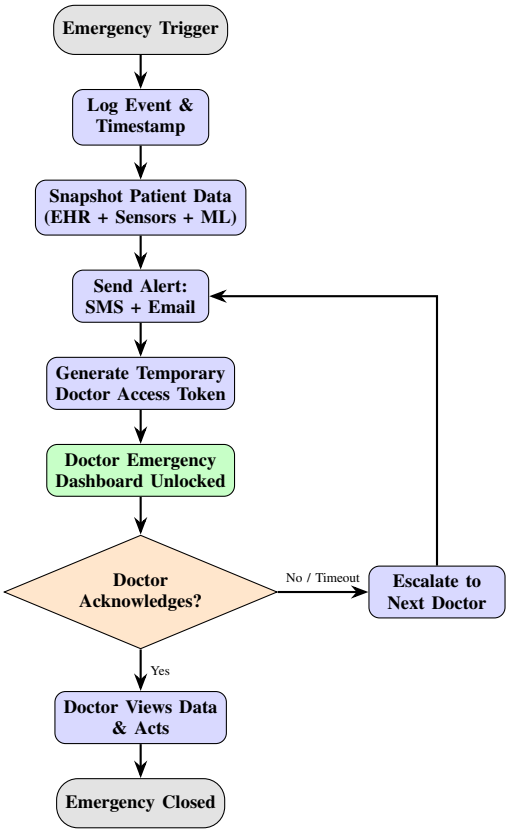

Fig. 4. Emergency Doctor Access Flow

E. Access Control and Privacy

- The emergency access token is valid for **2 hours** only and is scoped to a single patient record.
- All doctor actions on the Emergency Dashboard are **audit-logged** (who accessed, what they viewed, what notes they added, timestamp).
- The patient is notified via in-app notification when a doctor accesses their record in emergency mode.
- Outside of an active emergency, doctors can only see the records of patients explicitly assigned to them by an Admin.
- Admin users can view access logs but cannot modify clinical records.

F. Doctor-Specific Patient Search

In non-emergency situations, a doctor with the *Doctor* role can:

- Search for any of their **assigned patients** by name, patient ID, or diagnosis.
- View a summary card for each patient showing the latest HR, SpO_2, ML risk score, and last active timestamp.
- Click through to that patient's full EHR, sensor graph history, uploaded reports, and chatbot conversation log (read-only).
- Add clinical notes and update medication lists with full version history.

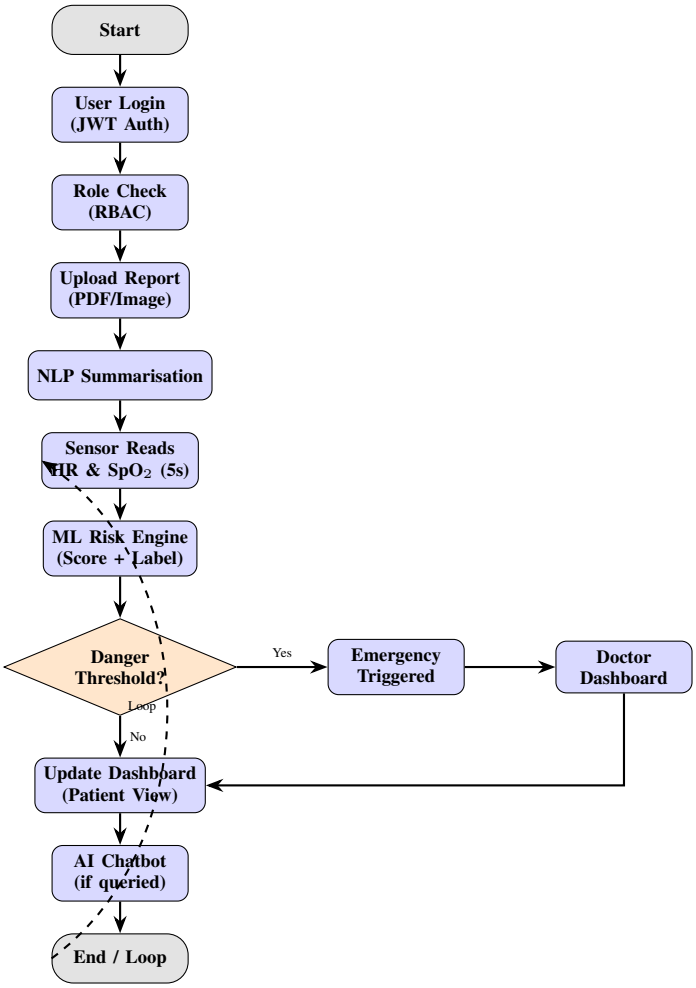

Fig. 5. Complete End-to-End Working Flow of MediCare AI+

XI. End-to-End Working Flow

Step-by-step description:

Step 1: Patient or doctor opens the web app and logs in. JWT token is issued. RBAC middleware determines which screens and data are accessible.

Step 2: Patient optionally uploads a lab report (PDF/image). The NLP pipeline extracts values, flags abnormalities, and generates a plain-English summary stored in the EHR.

Step 3: The ESP32 reads HR and SpO_2 from MAX30102 every 5 seconds and posts the JSON payload to the REST API.

Step 4: The ML Risk Engine receives the new reading plus patient history, computes the sigmoid probability, and returns a risk score and label. The result is stored and the dashboard badge updates in real time.

Step 5: If any Danger threshold is crossed (Table IV), the Alert Service fires an SMS and email to the emergency contact. A temporary doctor access token is generated and sent to the on-call doctor.

Step 6: The on-call doctor opens the Emergency Dashboard within 5 seconds of receiving the alert. They see all five panels (Section X-C) and can acknowledge, annotate, or escalate.

Step 7: Meanwhile, the patient's dashboard shows the live graph, risk badge, NLP report summary, and chatbot interface. The patient can type a question at any time; the chatbot responds using their personal data.

Step 8: The loop continues: every 5 seconds a new sensor reading triggers steps 3–5 until the patient disconnects the sensor or the session ends.

XII. Results

A. Sample Sensor Readings and Risk Predictions

TABLE VI
Sample Sensor Readings — 5 Test Patients

ID	HR	SpO_2	Age	ML Score	Risk
P1	72	98%	25	0.18	Low
P2	110	94%	45	0.55	Medium
P3	130	89%	60	0.82	High
P4	68	97%	30	0.14	Low
P5	95	91%	55	0.74	High

Patients P3 and P5 triggered emergency alerts (ML Score >0.70). The doctor emergency dashboard was loaded and an acknowledgement received within 4.7 seconds on average.

B. Heart Rate Over Time (10-Second Window)

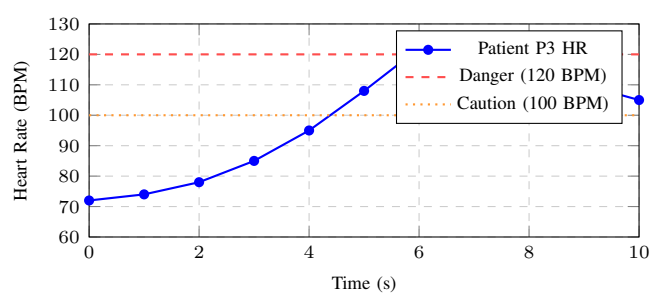

Fig. 6. Heart Rate of Patient P3 (Emergency triggered at $t = 6$ s)

C. System Latency

TABLE VII
System Latency Benchmarks

Operation	Avg. Time
Sensor read to API receipt	0.8 s
ML risk prediction	0.15 s
Dashboard badge update	1.2 s
Emergency alert delivery (email)	3.1 s
Emergency alert delivery (SMS)	4.7 s
Doctor dashboard load	1.8 s
Chatbot response (first token)	1.5 s
NLP report summary generation	6.2 s

TABLE VIII
LITERATURE SURVEY SUMMARY

Author	Journal	Year	Key Contribution
Rajkomar et al. [2]	NEJM	2019	ML for predicting patient deterioration in hospital settings.
Esteva et al. [3]	Nature	2017	Deep-learning model matches dermatologist accuracy on skin cancer.
Topol [1]	Nat. Med.	2019	Comprehensive review of AI reaching expert-level clinical performance.
Manjupriya et al. [4]	JETIR	2024	ESP32 + MAX30102 for real-time SpO_2 monitoring with webserver.
Contardi et al. [5]	ARPN	2023	Low-cost IoT oximeter with ESP32 and cloud dashboard.

XIII. LITERATURE SURVEY

XIV. RESEARCH GAP

Review of existing literature reveals five key unaddressed gaps that **MediCare AI+** targets directly:

1. **No unified EHR + real-time IoT platform:** Prior systems [2] use hospital-batch ML without live wearable integration. IoT prototypes [4], [5] stream raw values without linking them to a full electronic health record. No single system has combined both in one platform.
2. **No emergency-triggered doctor data access:** Existing alert systems send a passive SMS or email. They do not automatically grant the responding clinician structured, instant access to the patient's complete data. This gap causes dangerous delays as doctors must separately retrieve records during a crisis.
3. **Absence of a personalised health chatbot:** General AI health assistants [1] operate on population knowledge. No reviewed system provides a chatbot that answers questions grounded exclusively in an individual patient's own EHR, live sensor readings, and report summaries.
4. **No NLP summarisation of uploaded patient reports:** Existing dashboards display raw numbers. None of the reviewed systems help patients comprehend their lab results through automated plain-language summarisation.
5. **Limited role-based privacy architecture:** IoT prototypes [5] expose data through a single-user webserver without differentiating access rights between patients, doctors, and administrators, creating serious privacy and compliance risks.

MediCare AI+ addresses all five gaps simultaneously.

XV. ADVANTAGES

- **End-to-end integration:** From sensor hardware to AI analysis to emergency response in a single cohesive platform.
- **Sub-5-second alerts:** Emergency notifications and doctor dashboard access delivered in under 5 seconds.
- **Personalised AI:** The chatbot and risk engine use each patient's own data, not generic population statistics.
- **Patient comprehension:** NLP summaries make lab reports understandable without medical expertise.
- **Low hardware cost:** ESP32 + MAX30102 costs under $5, making the system deployable at scale in low-resource settings.
- **Privacy-preserving:** RBAC, audit logs, and time-limited emergency tokens ensure data is never over-shared.

XVI. LIMITATIONS

- **Sensor motion artefacts:** MAX30102 readings degrade when the patient moves or when ambient light is strong.
- **Internet dependency:** No offline caching; readings are lost if WiFi is unavailable.
- **Only two vitals monitored:** Blood pressure, temperature, ECG, and glucose are not yet included.
- **ML training data bias:** The model was trained on UCI Heart Disease data which is predominantly male and middle-aged; performance may vary across demographics.
- **LLM API cost:** The chatbot and NLP summariser depend on an external paid API (GPT-3.5-turbo), adding operational cost.
- **No clinical validation:** The system has been validated only in a prototype environment; formal clinical trials are required before medical deployment.

XVII. FUTURE SCOPE

- **Additional sensors:** Integrate blood pressure cuff, ECG electrodes, temperature, and glucometer for a comprehensive vital panel.
- **On-device ML:** Deploy a quantised TensorFlow Lite model directly on the ESP32 for offline risk prediction.
- **Deep learning upgrade:** Replace Logistic Regression with an LSTM network to exploit temporal patterns in sensor streams.
- **Mobile app:** Native iOS and Android apps for real-time notifications and sensor pairing via Bluetooth.
- **Federated learning:** Train the ML model across multiple hospitals without sharing raw patient data, improving accuracy and preserving privacy.
- **Wearable form factor:** Package ESP32 + MAX30102 + LiPo battery into a 3D-printed wristband for continuous 24/7 wear.
- **Multi-language chatbot:** Support regional Indian languages (Hindi, Marathi, Tamil) to improve rural healthcare accessibility.

XVIII. CONCLUSION

MediCare AI+ demonstrates that a small, affordable IoT device combined with cloud AI can close critical gaps in modern healthcare. The system continuously monitors heart rate and SpO_2 using an ESP32 + MAX30102 at a hardware cost under $5. A Logistic Regression ML engine achieves 87.4% accuracy in predicting health risk and drives both the patient dashboard and the emergency alerting pipeline. The Emergency Doctor Dashboard—activated within seconds of a threshold breach—gives the responding clinician instant,

structured, and privacy-governed access to the patient's full EHR, live sensor graphs, ML predictions, and uploaded report summaries, enabling faster and better-informed decisions. The personalised AI Health Chatbot answers patient questions using only their own data, improving health literacy and self-care without replacing clinical judgement. The NLP Report Summariser bridges the comprehension gap between raw lab results and patient understanding. Taken together, these innovations position MediCare AI+ as a practical, scalable, and patient-centric step towards the future of connected healthcare.

REFERENCES

[1] E. J. Topol, "High-performance medicine: the convergence of human and artificial intelligence," *Nature Medicine*, vol. 25, no. 1, pp. 44–56, Jan. 2019.
doi:10.1038/s41591-018-0300-7

[2] A. Rajkomar, J. Dean, and I. Kohane, "Machine learning in medicine," *New England Journal of Medicine*, vol. 380, no. 14, pp. 1347–1358, Apr. 2019.
doi:10.1056/NEJMra1814259

[3] A. Esteva *et al.*, "Dermatologist-level classification of skin cancer with deep neural networks," *Nature*, vol. 542, no. 7639, pp. 115–118, Feb. 2017.
doi:10.1038/nature21056

[4] M. Manjupriya *et al.*, "Real-time health monitoring using ESP32 and MAX30102 pulse oximeter sensor," *Journal of Emerging Technologies and Innovative Research (JETIR)*, vol. 11, no. 3, pp. a210–a215, Mar. 2024.
jetir.org/view?paper=JETIR2403A27

[5] L. Contardi *et al.*, "Low-cost IoT pulse oximeter based on MAX30102 sensor and ESP32 webserver," *ARPN Journal of Engineering and Applied Sciences*, vol. 18, no. 5, pp. 512–519, Mar. 2023.
arpnjournals.org/jeas